

# MATERI PRAKTIKUM STRUKTUR DATA

## Merge Sort dan Quick Sort

Materi praktikum ini membahas implementasi algoritma sorting menggunakan Merge Sort dan Quick Sort dalam bahasa pemrograman Java. Mahasiswa akan melakukan pengujian menggunakan jumlah data tertentu dan membandingkan waktu eksekusi kedua algoritma.

### Tujuan Praktikum

- Memahami konsep Merge Sort dan Quick Sort
- Mengimplementasikan algoritma sorting dalam Java
- Membandingkan waktu eksekusi algoritma sorting
- Menganalisis pengaruh jumlah data terhadap performa program

### 1. Merge Sort

Merge Sort menggunakan metode divide and conquer, yaitu membagi data menjadi bagian kecil, mengurutkannya, lalu menggabungkannya kembali.

Kompleksitas:  $O(n \log n)$

### 2. Quick Sort

Quick Sort bekerja dengan memilih sebuah data sebagai pivot, kemudian membagi data berdasarkan nilai yang lebih kecil dan lebih besar dari pivot.

Kompleksitas rata-rata:  $O(n \log n)$

### Program Java Perbandingan Merge Sort dan Quick Sort

```
import java.util.Random;
import java.util.Scanner;

public class MergeQuickSort {

    public static void mergeSort(int[] arr, int left, int right) {
        if(left < right) {
            int mid = (left + right) / 2;

            mergeSort(arr, left, mid);
            mergeSort(arr, mid + 1, right);

            merge(arr, left, mid, right);
        }
    }

    public static void merge(int[] arr, int left, int mid, int right) {
        int n1 = mid - left + 1;
        int n2 = right - mid;

        int[] L = new int[n1];
        int[] R = new int[n2];

        for(int i = 0; i < n1; i++)
            L[i] = arr[left + i];
        for(int j = 0; j < n2; j++)
```

```

        R[j] = arr[mid + 1 + j];

int i = 0, j = 0;
int k = left;

while(i < n1 && j < n2) {
    if(L[i] <= R[j]) {
        arr[k] = L[i];
        i++;
    } else {
        arr[k] = R[j];
        j++;
    }
    k++;
}

while(i < n1) {
    arr[k] = L[i];
    i++;
    k++;
}

while(j < n2) {
    arr[k] = R[j];
    j++;
    k++;
}
}

public static void quickSort(int[] arr, int low, int high) {
    if(low < high) {
        int pi = partition(arr, low, high);

        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

public static int partition(int[] arr, int low, int high) {
    int pivot = arr[high];

    int i = low - 1;

    for(int j = low; j < high; j++) {
        if(arr[j] < pivot) {
            i++;

            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }

    int temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;

    return i + 1;
}

public static void main(String[] args) {
    Scanner input = new Scanner(System.in);
    Random random = new Random();

    System.out.print("Masukkan jumlah data: ");
    int n = input.nextInt();

    int[] dataAsli = new int[n];

    for(int i = 0; i < n; i++) {
        dataAsli[i] = random.nextInt(1000);
    }
}

```

```

    }

    int[] merge = dataAsli.clone();
    int[] quick = dataAsli.clone();

    long startMerge = System.nanoTime();
    mergeSort(merge, 0, merge.length - 1);
    long endMerge = System.nanoTime();

    long startQuick = System.nanoTime();
    quickSort(quick, 0, quick.length - 1);
    long endQuick = System.nanoTime();

    System.out.println("\n\n=== PERBANDINGAN WAKTU ===");

    System.out.println("Merge Sort : " + (endMerge - startMerge) + " ns");
    System.out.println("Quick Sort : " + (endQuick - startQuick) + " ns");
}
}

```

## Perbandingan Algoritma

| Algoritma  | Kelebihan        | Kekurangan            |
|------------|------------------|-----------------------|
| Merge Sort | Stabil dan cepat | Butuh memori tambahan |
| Quick Sort | Sangat cepat     | Worst case $O(n^2)$   |

## Tugas Praktikum

- Jalankan program Merge Sort dan Quick Sort
- Uji dengan jumlah data 10, 100, dan 1000
- Screenshot hasil output
- Bandingkan waktu eksekusi
- Buat kesimpulan algoritma yang paling cepat

## Kesimpulan

Merge Sort dan Quick Sort merupakan algoritma sorting yang lebih efisien dibanding sorting dasar. Keduanya menggunakan konsep divide and conquer dan memiliki performa yang baik untuk data berukuran besar.